Towards Perfect Code Generation: Iterative API-based Feedback Loops

Harman Preet Singh
852863316

Abstract

The abstract paragraph should be indented ½ inch (3 picas) on both the left- and right-hand margins. Use 10 point type, with a vertical spacing (leading) of 11 points. The word **Abstract** must be centered, bold, and in point size 12. Two line spaces precede the abstract. The abstract must be limited to one paragraph.

Keywords: keyword 1 · keyword 2 · keyword 3

1 Introduction

Large Language Models (LLMs) are increasingly used in the field of software development for the generation of code, including HTML. These models are very capable of producing syntactically valid HTML that renders correctly in web browsers. However, there is a significant blind spot: while the generated code may be syntactically correct, it often lacks semantic correctness. In other words, the code functions as intended, but it may not adhere to best practices.

This potential non-adherence to semantic best practices is often seen in the overuse of the `<div>` tag, a phenomenon colloquially referred to as "divitis". Instead of using semantically meaningful tags in layout and content structure, such as `<section>` to denote sections or `<article>` for articles, developers may default to using `<div>` tags. For example, a developer might wrap an entire article in a `<div>` instead of using the more appropriate `<article>` tag. LLMs are trained on real code snippets, which often contain this pattern, making them likely to reproduce it in their generated output. Additionally, if a codebase already contains a significant amount of "divitis" code-generation tools like Claude Code may produce new code that perpetuates this issue.

There are two real consequences. Code that is hard to maintain, as the semantic meaning of the content has become an afterthought. But more importantly, users with visual impairments who rely on screen readers are negatively affected because screen readers use the semantic structure of a webpage to navigate them. A lack of semantic structure can make it difficult for these users to understand the content and context of a page.

Instructing an LLM to use semantic tags through a system prompt or explicit instructions within the prompt itself does not guarantee compliance, particularly for smaller or local models. There is a need for a reliable automated mechanism that can catch and correct this class of semantic errors in LLM-generated HTML.

We propose a solution in the form of an iterative validation-and-regeneration feedback loop. In this approach, the LLM-generated HTML is first validated using a validator tool which can flag semantic issues. The LLM is then reprompted with the identified errors, allowing it to correct its output. This loop continues until the output is free of errors or a maximum number of iterations is reached.

In this report, we aim to find out (RQ1) to what extent an iterative feedback loop can improve the semantic correctness of LLM-generated HTML code, and (RQ2) whether this guardrail approach can enable smaller, cheaper LLMs to achieve semantic correctness comparable to larger models. The remainder of this report is structured as follows: Section 2 reviews related work, Section 3 presents the research questions and hypotheses, Section 4 details the methodology, Section 5 describes the experimental setup, Section 6 presents the results, and Section 7 discusses the findings and implications. Finally, we conclude with a summary of our contributions and potential future work.

Preprint.

2 Related Work

Related work spans two interconnected areas: (1) LLM code generation quality, and (2) iterative self-correction and feedback loops. In the first part, we discuss what the literature says about LLMs generating code: their strengths, but also the documented tendency to reproduce patterns from training data including bad practices. In the second part, we cover approaches where LLMs are given feedback and asked to revise their output, particularly for Answer Set Programming (ASP) code. Finally, we review what is known about HTML semantic quality as a measurable property and the use of automated validation as a guardrail.

The literature on LLM code generation highlights both the strengths and limitations of these models. While LLMs can autonomously generate functioning code, they are also prone to reproducing patterns from their training data, including suboptimal or even harmful coding practices. For instance, Souly et al. [1] demonstrate that LLMs can be susceptible to poisoning attacks, where maliciously crafted inputs lead to the generation of flawed code. They show that even a small amount of poisoned data can significantly degrade the model's performance.

Tambon et al. [2] examined samples of 333 bugs collected from code generated using three leading LLMs and identified 10 common bug patterns, not including our focus on semantic quality. Dou et al. [3] evaluated several models on the length and complexity of generated code, finding that LLMs face challenges in generating successful code for more complex problems. They propose a novel training-free iterative method where the model gains the ability to self-critique and correct its own code, resulting in a repair success rate of 29.2% after two iterations. This study particularly is very relevant to our work, as it demonstrates the potential of iterative feedback loops in improving code quality. However, it primarily focuses on functional correctness rather than semantic or stylistic correctness, which is a gap that our research aims to address.

Addressing the second area of related work, several studies have explored iterative feedback and self-correction loops for LLMs. Ravi et al. [4] present a method that employs multiple feedback models to iteratively refine code generated by LLMs. Their approach demonstrates that iterative feedback can significantly enhance code quality, but it also introduces potential biases from the feedback models themselves. In contrast, our approach does not use other LLMs as feedback sources; instead, we employ an external, deterministic validator to provide feedback. This choice mitigates the risk of introducing additional biases and allows for a more objective assessment of code quality.

Madaan et al. [5] introduce 'Self-Refine', a self-refinement framework where the model critiques its own output and iteratively improves it. They tested the framework out on 7 diverse tasks, ranging from dialog response generation to mathematical reasoning, and showed that the generated code improves by 20% on average in task performance. The proposed method uses a single LLM as the generator, refiner and the feedback provider, whereas our approach separates the feedback mechanism from the generation process.

Much research has been conducted comparing the performance of smaller versus larger LLMs on code generation tasks. Coignion et al. [6] show that larger models generally outperform smaller ones in terms of code quality and functional correctness. However, they note that actual runtime execution efficiency of the generated code does not vastly differ, that is, smaller models can still produce code that runs efficiently.

Historically, treating semantic correctness as a measureable, objective property has been difficult because 'meaning' is highly contextual. However, some prior work has attempted to quantify semantic correctness in code. One example is the 'Generic-to-Semantic Ratio' (R_{gs}), which measures the count of non-semantic structural nodes (`<div>`, `<span>`) against native layout elements (`<header>`, `<nav>`, `<main>`, `<section>`, `<footer>`). A lower ratio indicates higher structural semantic quality. The 'Heading Hierarchy Traversal' (H_{score}) method measures structural correctness by scanning a webpage for skipped heading levels (e.g., a jump from `<h1>` directly to `<h3>`, skipping `<h2>`). These approaches provide a way to quantify semantic quality, but they are limited in scope and do not capture the full semantic intent of the code. In comparison, the W3C Markup Validation Service is a widely used tool that checks for syntactic and structural conformance to HTML standards. It is capable of identifying errors such as unclosed tags, incorrect nesting, and invalid attributes. However, it does not assess the semantic intent of the code, which is a critical aspect of web accessibility and user experience. This gap highlights the need for more comprehensive evaluation methods that can capture both syntactic correctness and semantic quality in HTML code. Our work does not address this gap, instead we acknowledge the need for further research in this area.

No prior work combines an external deterministic validator, an iterative reprompting loop, and a cost-efficiency comparison across model sizes in the context of HTML semantic quality. Our research aims to fill this gap by providing a comprehensive evaluation of how iterative feedback loops can improve the semantic quality of code generated by LLMs. By using an external validator, we can objectively assess the quality of the generated code without introducing additional biases. Additionally, our study will explore the cost-efficiency of using smaller models in comparison to larger ones.

3 Research Questions & Hypotheses

We aim to answer two research questions:

1. **RQ1:** Does the iterative API-based feedback loop improve the quality of code generation? We operationalise "quality" as the reduction in the number of errors in the generated code, measured before and after applying the feedback loop.
2. **RQ2:** Does this guardrail approach enable smaller, cheaper LLMs to achieve comparable quality to larger, more expensive models? We define "comparable quality" as equivalent error reduction rates, and "cost" is measured via mean generation time per prompt per model. Cloud-based models are explicitly out of scope for this study, as all models are run locally through Ollama, ensuring consistent evaluation conditions and avoiding external dependencies.

To evaluate our feedback loop, we also test the models under a blind condition, where the regeneration is performed without detailed feedback. This allows us to isolate the effect of the feedback loop.

We expect the feedback condition to outperform the blind condition across all models, as the iterative feedback loop provides targeted guidance for error correction. Furthermore, we hypothesise that reasoning-capable models (i.e., models that think and reason through the problem before generating code) will converge faster regardless of parameter count. Additionally, we expect that difficult prompts will require more iterations to achieve error-free code generation.

4 Methodology

4.1 System Architecture

The system consists of three key components. First, we have the language models themselves, which are served locally and ran via Ollama. The second and most important component, the validator, is the Nu Html Checker (vnu) service. This service is available via a public API and also, conveniently, as a Docker container [1]. Finally, the third component is the Python pipeline that handles the connection between the LLM and the validator.

Each of these components is its own distinct module. This modularity is a deliberate design choice, allowing for the LLM and validator to be swapped out without needing to rewrite the pipeline. We run the validator locally in Docker rather than using the public API for two reasons. First, the public API has rate limits that would make large-scale batch runs across 51 prompts, multiple models, and multiple conditions impractical. Second, running the validator locally allows us to pin the version and ensure consistency across all runs. This is important to ensure reproducibility of our results.

The pipeline, which is illustrated in Figure 1, operates in a loop. First, the prompt is sent to the LLM, which generates the initial HTML code. This code is validated by the validator. If any issues are found, the pipeline reprompts the LLM with the original code and the list of issues. This loop continues until either the code passes validation or the maximum number of iterations is reached.

4.2 Prompt Dataset

The prompt dataset was hand-crafted rather than sampled from an existing benchmark, because no such benchmark exists for HTML semantic quality specifically. The goal was to create tasks that vary in difficulty structurally.

We organise the prompts into a three-tiered structure: simple, medium, and difficult. The difficulty of a prompt is based on the structural complexity of the desired output. Simple prompts involve single components, such as a couple headers that contain some text. Medium prompts involve multi-section layouts, such as a navigation bar, a main content area, and a footer. Difficult prompts involve complex structures, such as the periodic table of elements, which requires a complex table layout with many containers to fit the name of the chemical element, its symbol, its atomic number, and other information. The more complex the structure, the more opportunities there are for the model to make a semantic error.

All models are evaluated on the exact same 51 prompts in the same order. We opted for this fixed design to ensure that differences in results are attributable to the model or condition, rather than to prompt sampling variance.

[1] ghcr.io/validator/validator:latest

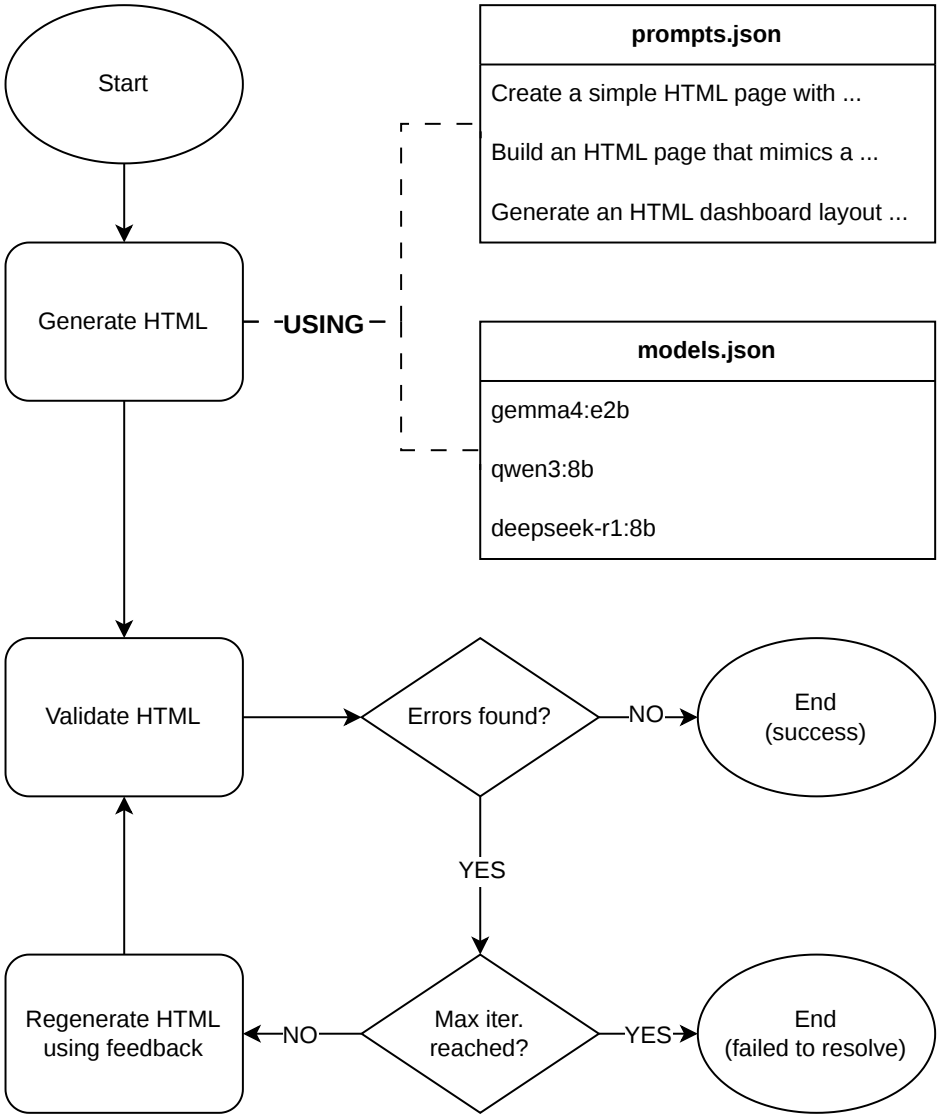

Figure 1: Overview of the iterative validation and regeneration loop.

4.3 Models & Configurations

We selected a range of models that vary in both parameter count and reasoning capability. Additionally, all models are available through Ollama and run locally at full precision (no quantisation was used), which keeps the hardware environment constant. An interesting finding from our initial testing was that reasoning capability matters more than parameter count. Smaller models with reasoning capability outperformed larger models without it. This finding shaped our final model selection, as we wanted to ensure that both axes (parameter count and reasoning capability) were represented in our experiments. An overview of all evaluated models is provided in Table 1. The table includes the model name, parameter count, reasoning capability (yes/no)

Table 1: Overview of all evaluated models

Model	Parameters (B)	Reasoning
gemma3:1b	1	No
qwen3:4b-thinking	4	Yes
deepseek-coder:6.7b	6.7	No
gemma4:e4b	8	Yes
qwen2.5-coder:14b	14.8	No
codestral:22b	22	No

We set the maximum iteration count as a parameter of interest. For each (model, prompt, condition) combination, we run the pipeline up to the maximum number of iterations, recording when convergence occurs. This allows us to analyse diminishing returns and determine the practical cut-off beyond which additional iterations yield negligible improvements. This is important for understanding the cost-quality tradeoff in RQ2, as it tells us how many iterations are really necessary.

4.4 Validation Strategy

The validator checks structural and syntactic conformance to the HTML specification. A limitation of the validator is that it cannot evaluate semantic intent. For example, it cannot tell you that `<div class="nav">` is worse than `<nav>` because both are syntactically valid.

Despite this limitation, the validator is still a meaningful signal. It can still detect many structural issues such as improper nesting, missing landmark elements, and attribute misuse. While error/warning count reduction is a necessary but not sufficient proxy for semantic improvement, it is still useful and can be triangulated with other measures.

In addition to running the pipeline in the feedback condition, we also run it in a blind condition. In the blind condition, the model is simply told to review and improve its output, with no error details provided. This allows us to isolate the contribution of the validator feedback itself, distinguishing it from the general benefit of giving a model a second attempt.

In related work where code quality was evaluated, the validator used was other LLMs, which struggle to provide deterministic outputs. To ensure that the validator is deterministic, we run a check on the same HTML input multiple times to verify that the validator produces identical results. This is non-trivial. If the validator were stochastic, our improvement metrics would be meaningless. We found that the validator is indeed deterministic, which gives us confidence in our before/after deltas.

4.5 Cost Metric

Since all models run locally via Ollama, there is no API pricing to measure. Instead, we proxy cost through three values captured per iteration: `gen_time_s` (wall-clock generation time), `prompt_eval_count` (tokens in the prompt), and `eval_count` (tokens generated). These together give a picture of both time cost and computational load per run.

We acknowledge that no cloud-hosted large model (e.g., GPT-4o, Claude) is included in this study. The comparison is therefore entirely within the local model ecosystem. This is a deliberate scope boundary, as cloud API costs introduce pricing variables outside our control. We note this as a direction for future work.

5 Experimental Setup

As mentioned in subsection 4.3, prior to the full experimental run, a small-scale pilot was conducted on a subset of prompts using a small variety of models. The most striking observation from this pilot was that models with native reasoning capabilities consistently resolved all validator errors within a single iteration, whereas non-reasoning models did not. Such models often were not able to remove all errors within the pre-defined iteration limit. This finding directly informed the decision to treat reasoning capability as an explicit variable in the final design, rather than focusing solely on parameter count. Furthermore, the pilot revealed a practical issue with local model outputs: models wrapped their HTML outputs in markdown code fences due to how these models are trained to generate outputs. Occasionally, the models would prefix responses with explanatory prose, which the validator would reject as incorrect HTML syntax. For this reason, for the full experimental run we introduce a module which strips such formatting artefacts before validation so that presentation-level slip-ups are not conflated with HTML errors. To conclude, the pilot served a dual purpose: it validated the core loop design and revealed the preprocessing steps necessary for a fair before/after comparison

6 Results

6.1 Guardrail Effectiveness

The error counts reported here are derived from the validator's output, which is a proxy for semantic correctness rather than a direct measure of it. As previously mentioned, while the validator provides a useful automated assessment of HTML validity, it may not capture all aspects of semantic correctness. We report error and warning counts as our primary metrics, but the results should be interpreted with this caveat in mind.

6.1.1 Overall before/after results (feedback condition)

Overall, across all models and prompts in the feedback condition ($n = 553$ runs), we observed a significant reduction in both errors and warnings after applying the guardrail. The aggregate results are summarized in Table 2. The mean error count decreased from 17.16 to 4.07, representing a 76.3% reduction, while the mean warning count decreased from 6.00 to 1.49, representing a 75.2% reduction. Paired Wilcoxon signed-rank tests confirmed that these reductions were statistically significant ($p < 0.001$), with 95% confidence intervals indicating meaningful decreases in both metrics.

Table 2: Aggregate before/after comparison

Metric	Mean before	Mean after	Δ	% reduction	p	95% CI (Δ)
Errors	17.16	4.07	13.10	76.3%	< 0.001	[4.16, 22.03]
Warnings	6.00	1.49	4.51	75.2%	< 0.001	[0.01, 9.02]

6.1.2 Feedback vs. Blind Ablation

In Table 3, we present the mean error reduction (before $-$ after) by prompt difficulty tier and condition, split by whether the model has a native reasoning/thinking mode. Higher values indicate more errors were resolved. Notably, in each difficulty tier, the feedback condition consistently outperforms the blind condition for both reasoning and non-reasoning models. This suggests that the validator's specific error list is indeed contributing to meaningful improvements in code generation quality, beyond just reprompting.

Difficulty	Condition	Reasoning models	Non-reasoning models
Simple	Feedback	0.33 (n=30)	0.18 (n=67)
Simple	Blind	0.13 (n=30)	-0.21 (n=67)
Medium	Feedback	0.55 (n=73)	0.38 (n=141)
Medium	Blind	0.14 (n=73)	-0.27 (n=141)
Difficult	Feedback	1.86 (n=28)	-0.10 (n=61)
Difficult	Blind	-1.25 (n=28)	-0.46 (n=61)

Table 3: Mean error reduction (before $-$ after)

Across all difficulty tiers, the feedback condition shows a clear advantage over the blind condition, particularly for reasoning models. For example, in the difficult tier, reasoning models in the feedback condition resolved an average of 1.86 errors, while those in the blind condition actually saw an increase in errors (-1.25). Across simple and medium tiers, the blind condition does not result in an increase in errors, but it still underperforms the feedback condition. This suggests that the specific feedback provided by the validator is useful for guiding the model towards more correct code generation, especially for models with reasoning capabilities.

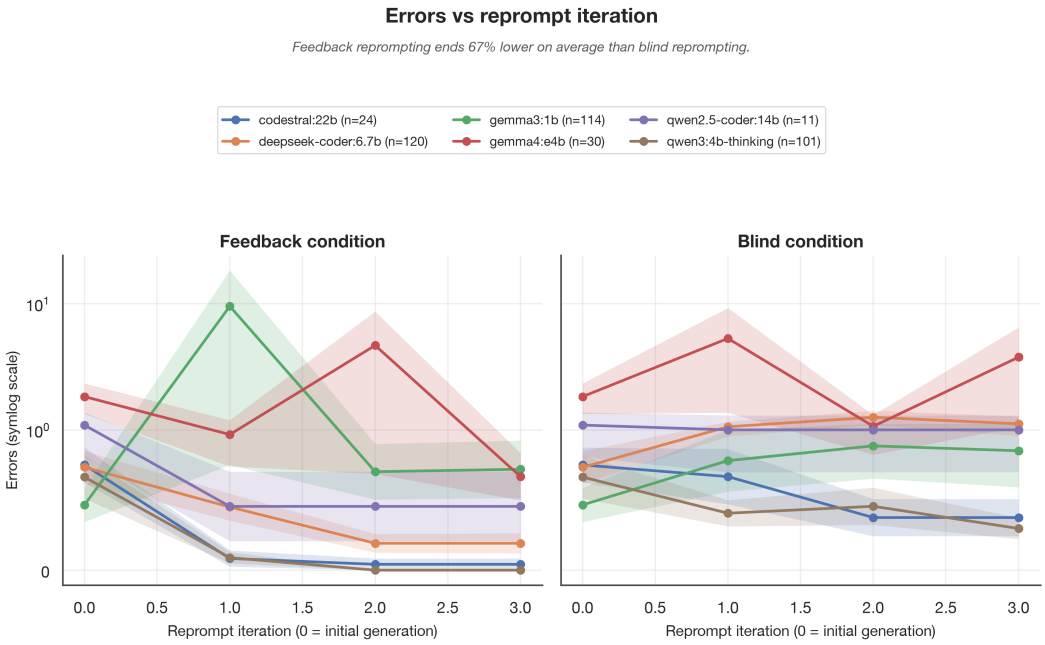

Figure 2: Mean error count per reprompt iteration, one line per model, feedback vs blind condition (shaded band = ± 1 SEM; symlog y-axis since per-model means span roughly 0–60 errors). Feedback and blind start from the same iteration-0 (initial) generation.

Discuss whether error reduction varies across simple / medium / difficult prompts. You would expect harder prompts to have higher starting error counts (because they simply have more lines of code to make a mistake in) and potentially require more iterations to converge – confirm or challenge this expectation with your data. A grouped bar chart per difficulty tier (before/after, per condition) works well here.

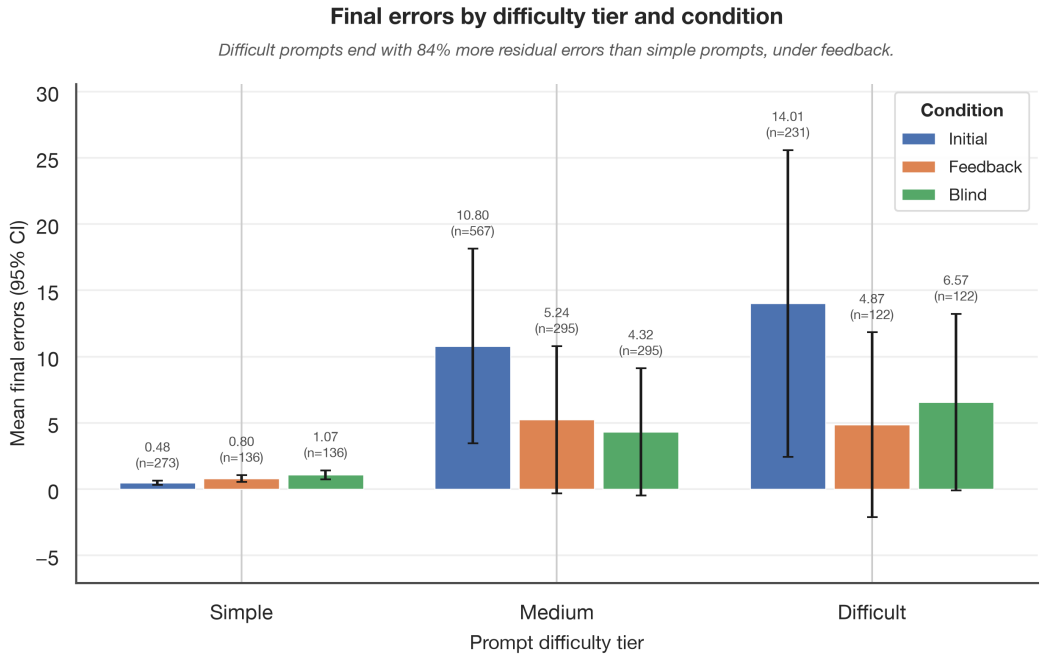

Figure 3: Mean final error count (95% CI) by prompt difficulty tier and condition. Value and n are annotated above each bar.

Discuss which error types the guardrail is most and least effective at resolving. For example: does it reliably fix missing-doctype errors? Does it struggle with disallowed-attribute errors? This dimension answers the qualitative question of what the guardrail actually corrects, and is important for contextualising the aggregate numbers. A small frequency table of error categories (before vs. after) serves this paragraph well. I think I can retrieve this data from the json files in validation/reprompt/, not sure though?

Table 4: Frequency of each W3C validator message category before (initial generation) and after (final feedback-condition iteration), across all models and runs. Categories are drawn from `util.validation.categorize_error` and combine both error- and warning-level messages; "other" is the uncategorised catch-all. Sorted by pre-reprompt frequency.

Category	Before	After	Δ	% resolved
other	11369	2619	8750	77.0%
disallowed-attribute	697	28	669	96.0%
stray-tag	324	100	224	69.1%
disallowed-child	314	225	89	28.3%
missing-title	61	32	29	47.5%
missing-doctype	18	29	-11	-61.1%
duplicate-tag	14	13	1	7.1%
cannot-recover	10	11	-1	-10.0%
missing-attribute	5	3	2	40.0%
missing-lang	0	1	-1	—

6.2 RQ2 — Cost-Efficiency

Briefly restate the scope: this is local-only comparison. Cost is operationalised as generation latency (`gen_time_s`) and token consumption (`prompt_eval_count` + `eval_count`) per run, not API pricing. Set the reader's expectation before presenting numbers

Present the main RQ2 finding: a scatter plot (or table) of each model's error reduction rate against its total generation time and token count. Colour-code or otherwise distinguish reasoning vs non-reasoning models. This is where the core finding lives – reasoning capability is more predictive of success than parameter count, with `codestral:22b` as the key counterexample (the most parameters, not the absolute best performance due to lack of reasoning -> a smaller model performed ever so slightly better).

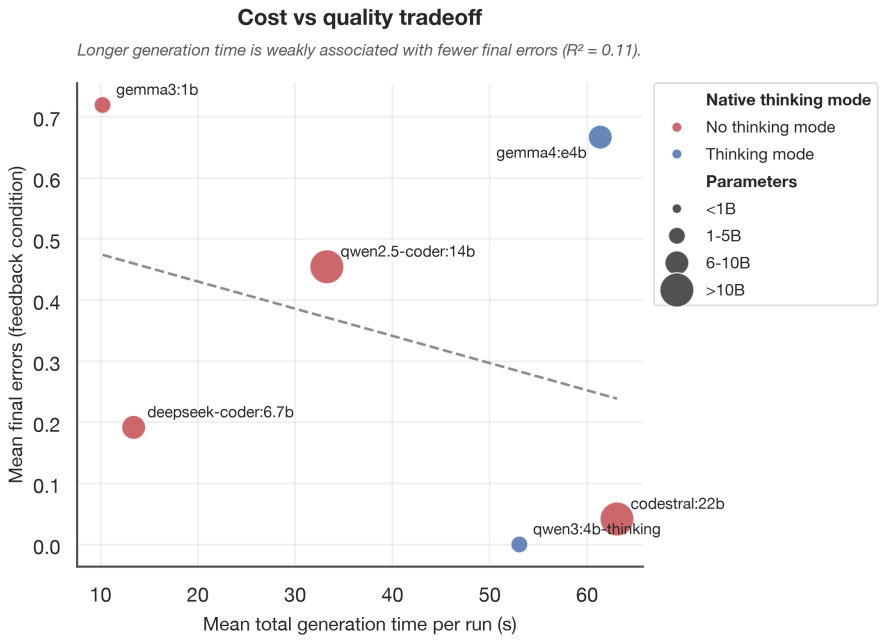

Figure 4: Mean total generation time per run vs mean final error count (feedback condition), one point per model. Colour encodes native thinking mode, point size encodes parameter-count bin.

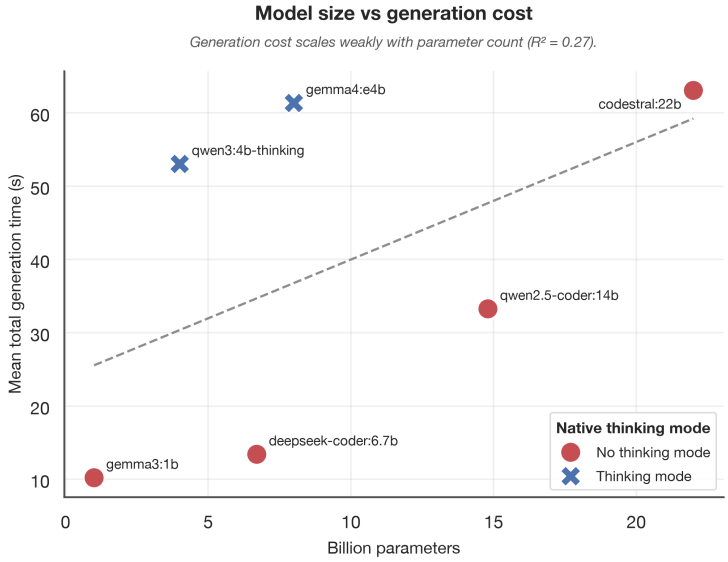

Figure 5: Parameter count vs mean total generation time per run, coloured by native thinking mode.

Give this finding its own paragraph because it is the most interesting and surprising result of RQ2. Explicitly compare a smaller reasoning model against codestral:22b. If a reasoning model with fewer parameters achieves better error reduction in fewer iterations at lower latency, that is a concrete, reportable conclusion that directly challenges the assumption that more parameters equal better output.

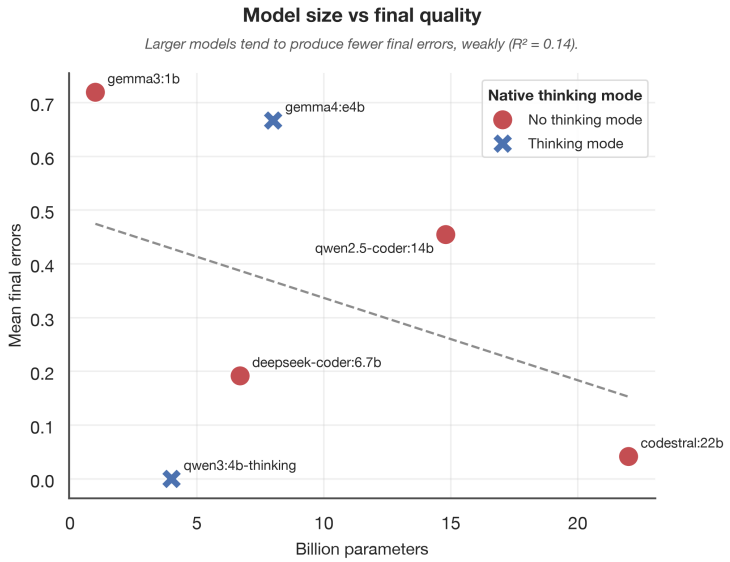

Figure 6: Parameter count vs mean final error count (feedback condition), coloured by native thinking mode. qwen3:4b-thinking (4B, reasoning) reaches a lower mean final error count than codestral:22b (22B, non-reasoning) at roughly a fifth of the parameters.

Does the `feedback` condition cost more than `blind` in token terms? It should, since it passes longer prompts (original code + error list). Is the extra cost justified by the quality gain? Frame this as a cost-benefit ratio: additional tokens spent per additional error resolved.

Table 5: Mean total tokens per run (initial generation plus all reprompt iterations) under feedback vs blind, per model. "Extra tokens" is feedback minus blind; "extra resolved" is the mean additional errors resolved by feedback over blind (positive = feedback resolves more). The last column is tokens spent per additional error resolved, only defined where feedback resolves strictly more errors than blind.

Model	Mean feedback tokens	Mean blind tokens	Extra tokens (feedback − blind)	Extra errors resolved	Tokens / extra resolved
All models (pooled)	9710	11293	-1584 (-14.0%)	-0.049	—
codestral:22b	1356	1998	-642 (-32.1%)	+0.333	-1925.4
deepseek-coder:6.7b	2703	3083	-380 (-12.3%)	+0.925	-410.6
gemma3:1b	2999	2528	+470 (+18.6%)	+0.132	3573.8
gemma4:e4b	8625	9172	-547 (-6.0%)	+3.100	-176.5
qwen2.5-coder:14b	3166	3855	-689 (-17.9%)	+0.545	-1262.8
qwen3:4b-thinking	6538	15584	-9046 (-58.0%)	+0.297	-30455.0
smollm2:135m	24292	23839	+453 (+1.9%)	-1.895	—

6.3 Iteration Count Analysis

Using the output of `stats.py`'s iteration-cutoff analysis, describe when models typically converge. Do most runs resolve within 1-2 iterations, or does error reduction continue meaningfully up to the maximum? Present a distribution (histogram or line chart) of iterations-to-convergence across all (model, prompt) pairs in the `feedback` condition.

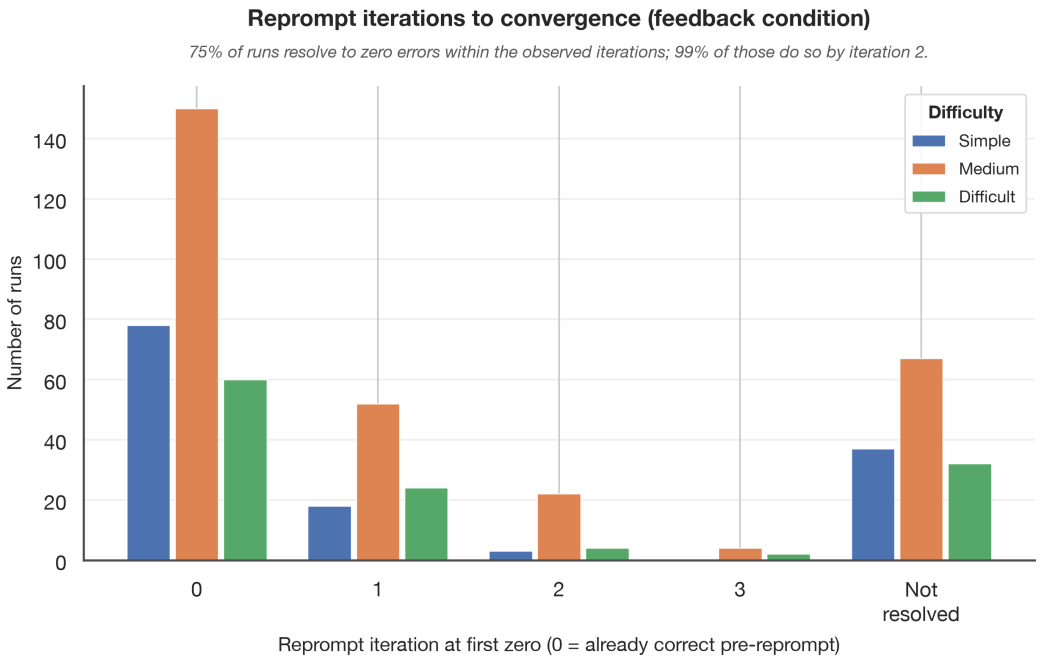

Figure 7: Distribution of the reprompt iteration at which each (model, prompt, trial) run first reaches zero errors, feedback condition, split by prompt difficulty tier. "Not resolved" = still nonzero at the last observed iteration.

Identify where the marginal gain per additional iteration drops below a meaningful threshold. If, for example, 80% of resolvable errors are fixed by iteration 2 and iterations 3-5 contribute minimally, that is your practical recommendation for the maximum iteration count. State this as a concrete finding rather than a vague observation. Note whether this threshold differs by model or difficulty tier.

Table 6: For each iteration cap, the percentage of feedback-condition runs already fully resolved (zero errors) and the mean remaining error count at that cutoff, by prompt difficulty tier. The per-model equivalent is available via `analysis.stats.iteration_cutoff_analysis(..., group_by="model")`.

Difficulty	Iteration cutoff	% resolved	Mean remaining	n runs
Simple	0	57.4%	0.97	136
Simple	1	67.6%	11.44	136
Simple	2	70.6%	0.80	136
Simple	3	70.6%	0.80	136
Medium	0	50.8%	20.76	295
Medium	1	61.7%	14.88	295
Medium	2	70.5%	9.85	295
Medium	3	70.8%	5.24	295
Difficult	0	49.2%	26.52	122
Difficult	1	62.3%	13.47	122
Difficult	2	66.4%	2.43	122
Difficult	3	66.4%	4.87	122

7 Discussion (2 – 3 pages)

8 Conclusion (0.5 – 1 page)

References

[1] Alexandra Souly, Javier Rando, Ed Chapman, Xander Davies, Burak Hasircioglu, Ezzeldin Shereen, Carlos Mougan, Vasilios Mavroudis, Erik Jones, Chris Hicks, Nicholas Carlini, Yarin Gal, and Robert Kirk. Poisoning attacks on llms require a near-constant number of poison samples, 2025. URL `https://arxiv.org/abs/2510.07192`.

[2] Florian Tambon, Arghavan Moradi-Dakhel, Amin Nikanjam, Foutse Khomh, Michel Desmarais, and Giuliano Antoniol. Bugs in large language models generated code: an empirical study. *Empirical Software Engineering*, 30, 02 2025. doi: 10.1007/s10664-025-10614-4.

[3] Shihan Dou, Haoxiang Jia, Shenxi Wu, Huiyuan Zheng, Muling Wu, Yunbo Tao, Ming Zhang, Mingxu Chai, Jessica Fan, Zhiheng Xi, Rui Zheng, Yueming Wu, Ming Wen, Tao Gui, Xipeng Qiu, and Xuanjing Huang. What is wrong with your code generated by large language models? an extensive study. *Science China Information Sciences*, 69, 01 2026. doi: 10.1007/s11432-025-4632-8.

[4] Ravin Ravi, Dylan Bradshaw, Stefano Ruberto, Gunel Jahangirova, and Valerio Terragni. Llmloop: Improving llm-generated code and tests through automated iterative feedback loops. 07 2025. doi: 10.1109/ICSME64153.2025.00109.

[5] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Sean Welleck, Bodhisattwa Majumder, Shashank Gupta, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback, 03 2023.

[6] Tristan Coignion, Clément Quinton, and Romain Rouvoy. A performance study of llm-generated code on leetcode, 07 2024.

Appendix

Full prompt dataset

Extended results tables

Flowchart (if not in main body)

Human evaluation rubric (if used)

Project timeline

- Full prompt dataset
- Extended results tables
- Flowchart (if not in main body)
- Human evaluation rubric (if used)
- Project timeline — your supervisor asked for this; an appendix is a natural place for if it doesn't fit in the main body